Week 13, Lecture 26, HW and Q

Week 13, Lecture 26, Homework and Weekly Quiz with Solutions.

Lecture 26: MLPs for Signal Classification

Homework

Problem 1: Tensor and layer shapes

An input minibatch contains 64 examples, each with 8 engineered DSP features. The first hidden layer has 16 neurons.

1. Give the input shape.
2. Give the shape of the weight matrix and bias vector for the first `Linear` layer.
3. Give the hidden activation shape.

Solution. Using row-major minibatches, the input is $\mathbf{X} \in \mathbb{R}^{64\times 8}$. A PyTorch `Linear(8,16)` stores a weight matrix with shape $(16,8)$ and a bias vector with shape $(16,)$. The output activation has shape $(64,16)$. Conceptually, for one example,

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}, \qquad \mathbf{W} \in \mathbb{R}^{16\times 8}.$$

Problem 2: Why ReLU matters

Suppose a network uses two linear layers with no nonlinear activation:

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{y} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2.$$

Show that the two layers are equivalent to one linear layer.

Solution. Substitute the first equation into the second:

$$\mathbf{y} = \mathbf{W}_2\mathbf{W}_1\mathbf{x} + \mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2.$$

Define

$$\mathbf{W}_{\text{eq}} = \mathbf{W}_2\mathbf{W}_1, \qquad \mathbf{b}_{\text{eq}} = \mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2.$$

Then

$$\mathbf{y} = \mathbf{W}_{\text{eq}}\mathbf{x} + \mathbf{b}_{\text{eq}}.$$

Therefore stacking linear maps alone does not create nonlinear decision boundaries. A nonlinear activation such as ReLU is required.

Problem 3: ReLU

For

$$\mathbf{z} = [-2,\ -0.5,\ 0,\ 1.2,\ 4]^T,$$

compute $\mathrm{ReLU}(\mathbf{z})$ and state the derivative away from zero.

Solution.

$$\mathrm{ReLU}(z) = \max(0, z),$$

so

$$\mathrm{ReLU}(\mathbf{z}) = [0,\ 0,\ 0,\ 1.2,\ 4]^T.$$

Away from zero,

$$\frac{d}{dz}\mathrm{ReLU}(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

Problem 4: Cross-entropy from logits

A three-class classifier produces logits

$$\mathbf{z} = [2.0,\ 1.0,\ 0.0].$$

The true class is class 0. Compute the softmax probability of the true class and the cross-entropy loss.

Solution.

$$p_0 = \frac{e^2}{e^2 + e^1 + e^0} \approx 0.6652.$$

Then

$$L = -\ln p_0 \approx 0.4076.$$

PyTorch `CrossEntropyLoss` accepts the raw logits directly; do not apply `softmax` first.

Problem 5: One gradient-descent update

A scalar parameter has current value $\theta_t = 1.5$, learning rate $\eta = 0.02$, and gradient $\partial L / \partial \theta = 4.0$. Compute the next value using gradient descent.

Solution.

$$\theta_{t+1} = \theta_t - \eta \frac{\partial L}{\partial \theta} = 1.5 - 0.02(4.0) = 1.42.$$

Problem 6: Training-loop reasoning

Explain why the following order is appropriate:

1. forward pass,
2. loss,
3. `optimizer.zero_grad()`,
4. `loss.backward()`,
5. `optimizer.step()`.

What happens if `zero_grad()` is omitted?

Solution. The forward pass creates predictions and a computational graph. The loss creates the scalar objective. `zero_grad()` removes gradients left from the previous optimization step. `backward()` computes current gradients. `step()` updates parameters using those gradients. If `zero_grad()` is omitted, PyTorch accumulates old and new gradients, so the optimizer uses unintended sums across minibatches or epochs.

Problem 7: DSP interpretation

A classifier uses eight standardized features: spectral centroid, zero-crossing rate, spectral roll-off, RMS energy, spectral flux, and three MFCCs. Explain why feature standardization is generally useful before an MLP.

Solution. Gradient-based optimization is sensitive to feature scale. Without scaling, a feature measured on a numerically large scale can cause disproportionately large activations and gradient contributions. Standardization gives features comparable numerical ranges and often improves optimization conditioning. Training-set statistics must be used so validation/test information does not leak into preprocessing.

Weekly Quiz: 10 minutes

Q1

A batch contains 32 examples, each with 8 DSP features. What is the input shape to an MLP whose first layer is `Linear(8,16)`?

Answer: $(32,8)$.

Q2

Why are two consecutive linear layers with no nonlinear activation equivalent to one linear layer?

Answer: The composition of affine maps is another affine map:

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2).$$

Q3

For multiclass classification with PyTorch `CrossEntropyLoss`, should the final network layer apply `softmax`?

Answer: No. The final layer should return raw logits. `CrossEntropyLoss` internally combines log-softmax with negative log-likelihood in a numerically stable way.

Q4

A sample has predicted probability $p_y = 0.8$ for its correct class. What is its cross-entropy loss?

Answer:

$$L = -\ln(0.8) \approx 0.223.$$

Q5

What does `loss.backward()` compute?

Answer: Gradients of the scalar loss with respect to upstream leaf parameters that require gradients, i.e. the relevant components of $\nabla_\theta L$.

Q6

What is the conceptual difference between `loss.backward()` and `optimizer.step()`?

Answer: `backward()` computes gradients. `step()` uses the optimizer's rule, such as Adam, to convert those gradients into parameter updates.